ChatGPT

Texas Hold'em Bot Strategies and an Integration Roadmap for poker-coach-alpha

Executive Summary

The short version is this: **`poker-coach-alpha` is currently a lightweight coaching and gameplay scaffold, not a serious game-theoretic poker bot**. The repository is built around FastAPI, WebSocket messaging, and PokerKit; its live play policies are a mix of `SimpleBot`, `TightBot`, and an `EquityBot` that compares estimated hand strength to required equity and then selects from a small legal action set. The LLM path is intentionally constrained: the README says the default mode is heuristic-only, and even when an LLM is enabled it is primarily used for explanations while action selection stays heuristic with safe fallback. The project plan explicitly lists depth-limited lookahead, Monte Carlo rollout EV, and CFR-lite as **future** work rather than current implementation. [1]

By contrast, the strongest documented no-limit Texas Hold’em systems are still overwhelmingly **solver-centric**. DeepStack used continual re-solving plus depth-limited lookahead and a learned value function. Libratus used an offline blueprint strategy, nested subgame solving, and a self-improver. Pluribus extended blueprint-plus-search ideas to six-player no-limit hold’em. ReBeL later showed that public-belief-state RL plus search could also achieve superhuman heads-up no-limit hold’em with less handcrafted domain knowledge than earlier poker AIs. [2]

That matters because it implies a very clear strategic gap. The provided repo does **not** currently maintain explicit combo-level opponent ranges, does not solve subgames online, does not train regrets offline, and does not benchmark exploitability or solver agreement. It does track basic human stats such as VPIP, PFR, and AFq, and its `DecisionContext` already leaves a placeholder for `villain_ranges`, so the architecture is **compatible** with stronger methods; it just has not crossed that line yet. [3]

The most practical upgrade path is therefore **hybrid, not revolutionary**. The highest-return steps are: move from shorthand ranges to combo-weighted ranges; add an opponent-model layer; add a rollout EV evaluator with caching; add a benchmark harness; then introduce depth-limited toy-tree search under a strict latency budget, optionally backed by an external solver such as TexasSolver, `postflop-solver`, Noam Brown’s river solver, or a commercial UPI-compatible engine like PioSOLVER. That path tracks what the repo’s own plan already points toward and aligns with how strong public systems are structured. [4]

My bottom-line recommendation is straightforward: **do not try to jump directly from the current repo to “Pluribus-like 6-max superhuman bot”**. Six-player no-limit hold’em was itself a major research milestone, and even the best open-source ecosystem around poker remains far stronger in heads-up or postflop subgame solving than in full six-player end-to-end play. Build a strong advisor / training bot first, then a strong heads-up or postflop-search bot, and only then consider deeper six-max blueprint work. [5]

Repository Baseline

What the current repository actually implements

The repository presents itself as a “simple, understandable scaffold” for a Texas Hold’em MVP on FastAPI, WebSocket, and PokerKit. The current layout is application-first: `app/` for FastAPI, `poker/` for the engine and bots, `ws/` for messaging schemas, and `tools/` for simulations. The Python requirements are web-framework and orchestration oriented, not solving oriented: `fastapi`, `uvicorn`, `pydantic`, `httpx`, `pokerkit`, `openai`, `aiohttp`, `tiktoken`, and utilities such as `jsonschema`. [6]

Its simplest policy is exactly what the name suggests. `SimpleBot` documents its preference order as **check > call > min raise_to > fold > anything**. That is a safety-first placeholder, not a strategic poker model. `BotManager` then seeds a fixed mix of `SimpleAsyncBot` and `TightBot` across seats 2–6, giving some seat variety but still keeping the bot pool stationary and shallow. [7]

The strongest native policy today is `EquityBot`, but it is still heuristic. It builds a `DecisionContext`, requests hand-strength analysis, computes `edge = hand_strength - required_equity`, applies street-specific thresholds, and then chooses among calls, folds, and a few raise candidates with light randomization. This is a sensible step above `SimpleBot`; it is also still fundamentally an **equity-threshold bot**, not a range-vs-range equilibrium bot. [8]

Action abstraction is already present, just in a very small form. In `engine.py`, postflop raise candidates are generated from fixed pot fractions of **one-third, one-half, two-thirds, one pot, and two pot**, plus all-in. The technical plan mirrors that design style by proposing a future advisor with sizes in `{0.33, 0.66, 1.0 pot, all-in}` and a latency budget below roughly 150–200 ms. That is exactly the kind of restricted toy tree strong systems also use — but in the repo today it is only a lightweight action menu, not a solving stack. [9]

Range modeling is also intentionally thin. The current `Range` type is a frequency-weighted shorthand map like `{"AKs": 1.0}`, and the default preflop range builder returns a conservative non-empty range with hands such as `AA`, `KK`, `QQ`, and `AKs`, slightly widened by position. `DecisionContext` has a `villain_ranges` field, but it is explicitly part of the “future extensions” area and is optional. In other words, the repo is architecturally range-aware, but not yet range-driven. [10]

The LLM layer is also more conservative than the project name might suggest. The README defines three modes: heuristic-only, LLM explanation plus heuristic actions, and fallback to heuristics on error. `LlmBot` is implemented as a wrapper around the existing analysis pipeline and uses a safe fallback policy whenever analysis fails or the LLM output is unusable. That is actually a good design choice for production stability, but it also means the repo should currently be compared to **heuristic/equity bots with an explanation layer**, not to genuine LLM-first decision agents. [11]

What that means against human opponents

Against casual or weaker human players, the repo’s bot family has some real practical strengths. It is fast, predictable to operate, easy to debug, rarely crashes into illegal moves because everything flows through PokerKit legality checks, and it at least uses pot-odds/equity-style reasoning rather than raw random play. That makes it a decent training scaffold, a decent coach UI foundation, and a plausible sandbox opponent for UX testing. [12]

Against competent humans, though, the weaknesses are structural. The opening and response ranges are too sparse, action sizes are too narrow, the opponent model is mostly absent, and the policy is stationary enough that a skilled player can quickly learn where it over-folds, over-checks, or uses obviously capped raising patterns. Strong poker AIs moved beyond this years ago by combining broader abstractions, explicit ranges, subgame solving, and real-time search. [13]

For that reason, the repository today is best understood as an **excellent integration target for better poker algorithms**, not as a finished poker algorithm in its own right. That is a favorable position to be in: the engine and messaging are already usable, and the roadmap already points toward the right technical direction. [14]

A direct baseline comparison

Dimension	`poker-coach-alpha` today	Strong mainstream bots
Core policy	Heuristic / equity-threshold decisions with safe fallbacks. [15]	Solver-centric: CFR-family blueprints, subgame solving, depth-limited search, or belief-based RL+search. [2]
Range modeling	Minimal shorthand ranges; `villain_ranges` is optional future extension. [10]	Explicit range distributions are central to strategy computation. [16]
Action abstraction	Small, hand-authored size set; legal-action based. [17]	Restricted but solver-designed abstractions paired with search or blueprint policies. [18]
Opponent adaptation	Basic VPIP/PFR/AFq tracking only. [19]	Stronger bots still usually aim for robust play, but their search stacks reason over opponent ranges and continuation values in much richer ways. [20]
Latency profile	Very low and production-friendly. [21]	Usually moderate to high unless inference is heavily cached or abstracted. [22]
Best role	Coach scaffold, explainer, sandbox opponent. [23]	Competitive bot or solver-backed advisor. [24]

Mainstream Strategy Taxonomy

The important thing to understand is that “poker bot” is not one thing. In practice, the field separates into solver-style equilibrium methods, search-heavy real-time methods, RL approximators, and lightweight heuristics. The latency and engineering ratings below are **engineering judgments inferred from the cited papers and official codebases**; exact performance depends heavily on abstraction size, game type, and hardware. [25]

Strategy family	Core algorithm	Resource needs	Typical documented human performance	Strengths	Weaknesses	Engineering complexity and online suitability
CFR variants	Traverse infosets, accumulate counterfactual regrets, average policies; CFR+, Linear CFR, and DCFR accelerate convergence.	High CPU/ RAM for large abstractions; best suited to offline training or smaller subgames.	Excellent when scaled well; this family underlies landmark poker systems and public strong bots.	Theoretically principled, robust, highly compatible with exploitability analysis.	Full-tree iterations are expensive; abstraction design matters a lot.	High complexity; online use is poor unless precomputed or combined with subgame solving.
CFR-lite and toy-tree solving	Solve a restricted handwritten tree with a few actions or buckets, often only around the current decision.	Moderate CPU; tractable under strict time budgets if the tree is tiny.	Useful for advisors and local approximation; not enough alone for world-class full-game play.	Practical, controllable, easy to insert into a live client.	Strong abstraction bias; brittle if ranges or size sets are poor.	Medium complexity; online suitability is good under ~100–300 ms with caches.
Monte Carlo CFR	Sample trajectories or opponents so each iteration touches only a fraction of the tree, while matching CFR updates in expectation.	Lower per-iteration cost than vanilla CFR; still heavy for large games.	Strong large-game workhorse in research and public solvers; often used for blueprints.	Scales better than full-tree CFR, especially with large abstractions.	Higher variance; tuning and pruning matter.	Medium-to-high complexity; better than full CFR for online fragments, but still mainly offline.
Depth-limited lookahead	Re-solve the current public state to limited depth, then use leaf values or continuation strategies at the frontier.	Moderate to very high depending on tree width and leaf evaluator.	This is a core ingredient in several superhuman systems.	Strong local adaptation, can react to off-tree actions, avoids solving the whole game online.	Needs a good blueprint or leaf evaluator; easy to get wrong.	High complexity; online feasible if trees are narrow and cached.

Strategy family	Core algorithm	Resource needs	Typical documented human performance	Strengths	Weaknesses	Engineering complexity and online suitability
MCTS and IS-MCTS	Search sampled information sets rather than full solved strategies.	Typically cheaper than full solver stacks at small scale, but can explode with imperfect information and large action spaces.	Mostly research-grade in poker; not the method behind the headline human-beating HUNL/ 6-max milestones.	Flexible online search, easy to prototype.	Weak theoretical guarantees in large imperfect-information poker; can mis-handle belief structure.	Medium complexity; online-suitable for toy settings, usually inferior to solver-centric methods in serious NLHE.
Model-based RL	Learn over public belief states and combine search with learned models; ReBeL is the clearest example.	Research-scale training; sizeable compute and infrastructure.	ReBeL reported superhuman HUNL with less domain knowledge than previous systems.	Reduces some handcrafted abstraction burden; unifies learning and search.	Hard to reproduce; difficult training, validation, and debugging.	Very high complexity; online inference/ search can be viable, training is expensive.
Policy/ value network methods	Learn regret, policy, or value approximators instead of or alongside tabular solving; includes Deep CFR and NFSP.	Usually GPU-friendly and memory-hungry; large replay or reservoir buffers.	Strong research results in large poker games; NFSP approached state-of-the-art limit Hold’ em performance, but these are not standard open-source drop-ins for full NLHE production.	Good amortized inference latency after training; can reduce handcrafted abstraction.	Training instability, data hunger, and weaker interpretability.	High complexity; online inference can be low-latency, but the training stack is heavy.
Heuristic and equity bots	Use hand strength, pot odds, thresholds, simple rules, and maybe small randomization.	Very low compute.	Adequate for demos or weak pools; far below solver-backed systems.	Fast, simple, debuggable, easy to deploy.	Easily exploitable, poor balance, weak adaptation.	Low complexity; excellent online suitability, poor ceiling.

Strategy family	Core algorithm	Resource needs	Typical documented human performance	Strengths	Weaknesses	Engineering complexity and online suitability
LLM-assisted agents	Use an LLM for action selection, explanation, plan decomposition, or policy distillation.	High latency/cost for naive prompting; lower if used only as explainer or offline labeler.	Not mainstream among top NLHE bots; recent poker-LLM work depends on poker-specific training and solver data rather than pure prompting.	Great for explanations, coaching, natural-language UX.	Weak reliability as a primary policy, expensive, hard to constrain.	Medium-to-high complexity; viable online mainly as an explanation layer, which is also how your repo currently treats it.

A useful way to compress that table into one sentence is this: **the strongest no-limit Hold'em bots are still built around solver thinking**, even when they add neural networks or RL. Neural networks usually help with value approximation, policy compression, or training efficiency; they do not replace the need to reason over hidden information, ranges, and equilibrium pressure. [35]

That is also why the current repository's design is simultaneously sensible and limited. It already has the parts you want in a product — engine, messaging, UI, logging, simulation script, explanation pathway — but it is missing the parts the research literature says matter most for strength: explicit ranges, subgame search, exploitability-aware evaluation, and blueprint policies. [36]

Open-source Landscape

The open-source poker ecosystem is useful, but it is not flat. Some projects are **research frameworks**, some are **engines**, some are **subgame solvers**, and some are **fuller poker bots**. For your repo, the best integration targets are the ones that either improve local decision quality with minimal disruption or give you rigorous evaluation tools. [37]

Project	What it implements	Maturity	Language and dependencies	License	Integration fit for `poker-coach-alpha`	Source
`poker-coach-alpha`	FastAPI/PokerKit coach scaffold; heuristic bots; LLM explanation/fallback.	Early prototype.	Python; FastAPI, PokerKit, OpenAI client, WebSocket stack.	MIT.	Baseline. Strong app shell, weak strategy core.	[6]

Project	What it implements	Maturity	Language and dependencies	License	Integration fit for `poker-coach-alpha`	Source
OpenSpiel	Universal poker environment plus CFR, CFR+, MCCFR, Deep CFR, NFSP, IS-MCTS, exploitability tooling.	Very mature and active.	C++ core with Python bindings; toolchain specifics depend on installation path.	Apache-2.0.	**High** for offline solving prototypes, exploitability, toy-tree evaluation, and algorithm benchmarking.	38
RLCard	Card-game environments with RL/search support; no-limit Hold'em human interface with abstracted action space; optional PyTorch install.	Mature research toolkit.	Python; default env install, optional `rlcard[torch]`.	MIT.	**Medium** for RL experimentation and offline evaluation; action abstraction mismatch limits direct live integration.	39
PyPokerEngine	Simple Python poker engine for AI development.	Mature but old-school.	Python; pip-installable.	MIT.	**Low to medium** because you already use PokerKit, which is a stronger fit for your current architecture.	40
PokerRL	Research framework for NFSP, RPG, Deep CFR, SD-CFR and distributed poker RL.	Research-mature, but dated stack.	Python plus C++ exports; Conda, Docker, PyCrayon, Ray, old PyTorch.	MIT.	**Medium** if you commit to a research branch; otherwise too heavy and dated for a near-term product path.	41
Deep-CFR	Scalable implementation of Deep CFR and Single Deep CFR in PokerRL.	Useful research reference, not turnkey.	Python; PokerRL, Conda, Docker, PyCrayon, PyTorch.	MIT.	**Medium** as a reference implementation for leaf/value learning; **low** as a quick dependency.	42

Project	What it implements	Maturity	Language and dependencies	License	Integration fit for `poker-coach-alpha`	Source
`slumbot2019`	CFR+ and MCCFR, endgame resolving, card and betting abstractions, head-to-head eval, BR tooling.	Solid solver codebase.	C++17 / gcc.	MIT.	**Medium-high** for heads-up solver concepts and abstraction pipelines; adaptation cost is nontrivial.	43
TexasSolver	Efficient Texas Hold’ em and short-deck GTO solver; cross-language calls; JSON strategy dumps; desktop/ console tooling.	Mature but sporadically maintained. Latest public release is older, but repo activity continued later.	C++.	AGPL-3.0.	**High technical fit** for postflop solving; **medium practical fit** because of AGPL and native-binary integration risk.	44
`postflop-solver`	Rust library for postflop solving using Discounted CFR; multithreaded; no abstraction; exploitability API.	Strong backend, but development suspended.	Rust; Rayon/ Bincode/Zstd features.	AGPL-3.0.	**High** if you want a library-style postflop engine and can accept AGPL + maintenance risk.	45
`noambrown/ poker_solver`	Modern river subgame solver with CFR, CFR+, MCCFR, Fictitious Play, DCFR; Python reference and optimized C++; JSON subgame format.	Early-stage but very relevant.	Python + C++ + CMake.	MIT.	**High** for incremental river integration, subgame serialization, and a modern CFR reference.	46

Project	What it implements	Maturity	Language and dependencies	License	Integration fit for `poker-coach-alpha`	Source
DecisionHoldem	HUNL bot with blueprint linear CFR plus safer depth-limited real-time search; scripts for Slumbot/ OpenStack/ human play.	Valuable research code; partially brittle and asset-heavy.	C++11 plus Python GUI scripts; external binary assets and blueprint files.	AGPL-3.0.	**Medium** as an architecture reference; **low to medium** as a direct dependency.	[47]

Which projects are worth evaluating first

If the goal is **practical strength gain with sane engineering risk**, I would prioritize the ecosystem in this order.

OpenSpiel first, because it gives you the cleanest route to exploitability measurement, toy-tree CFR baselines, universal-poker experimentation, and algorithm benchmarking without forcing you to replace your product shell. It is the single best "research harness" to bolt onto your existing repo. [48]

Noam Brown's river solver second, because it is modern, modular, MIT-licensed, already separates Python reference logic from optimized C++ code, and defines a JSON subgame format that maps well onto a service boundary. That makes it unusually attractive for incremental adoption. [46]

TexasSolver or `postflop-solver` third, if you want a real postflop engine quickly and can live with AGPL constraints. `postflop-solver` is the cleaner library surface; TexasSolver is the more user-facing and cross-language-friendly package. Both can materially raise postflop decision quality if you restrict the tree size. [49]

PokerRL / Deep-CFR belong later. They are useful if, and only if, you decide that long-term differentiation should come from learned value/policy approximators rather than solver-backed search. They are not the fastest road to a stronger advisor inside your current application. [50]

One more ecosystem fact matters a lot for your roadmap: **I did not find a mature open-source six-player full-game bot comparable to Pluribus among the publicly usable projects surveyed here**. The useful open-source surface is much richer in heads-up engines, universal research frameworks, and postflop solvers than in end-to-end 6-max no-limit bots. That is one more reason to pursue a hybrid advisor / search path first. [51]

Improvement Roadmap and Integration Plan

The table below is the concrete upgrade list I would use. The **effort, expected gain, compute, and risk columns are my engineering estimates**, informed by the cited systems and your current codebase.

Priority band	Improvement	Why it matters	Effort estimate	Expected strength gain	Compute estimate	Main risk	Basis
Short-term	Combo-level range engine and preflop chart priors	Your current default ranges are too sparse and shorthand-only. Better combo weights are the foundation for every stronger method.	1–2 weeks	High	CPU-only	Low	52
Short-term	Opponent model with action-conditioned range updates	You already track VPIP/PFR/AFq and have `villain_ranges` in `DecisionContext`; turn that into live weighted ranges.	1–2 weeks	High	CPU-only	Medium	53
Short-term	Benchmark harness and telemetry	Right now you can run LLM-vs-bot sims and log BB/100 to CSV; formalize that for all policies before adding more strategy code.	1 week	Very high ROI as infrastructure	CPU-only	Low	54
Short-term	Cached rollout EV evaluator	Gives you a real EV signal per legal action without jumping straight to full solving.	2–3 weeks	Medium-high	1–8 CPU cores depending on rollout count	Medium	55
Mid-term	Depth-limited toy-tree search under latency budget	This is the single most realistic jump from heuristic bot to solver-like advisor.	4–6 weeks	High	4–16 CPU cores for tuning; sub-200 ms target online	Medium-high	56
Mid-term	Solver cache keyed by public state, board, stack, ranges	Makes search economically viable online. Without caching, latency will dominate.	2–3 weeks	Medium	RAM-heavy but manageable	Medium	57

Priority band	Improvement	Why it matters	Effort estimate	Expected strength gain	Compute estimate	Main risk	Basis
Mid-term	External postflop solver adapter	Lets you use a real engine for flop/turn/river spots instead of building everything from scratch.	3–5 weeks	Very high postflop gain	16–64 GB RAM, native process management	Medium-high	58
Long-term	Offline blueprint training on abstractions	Needed if you want stronger priors than simple charts and live rollouts.	6–10 weeks	High	Tens of CPU cores, potentially hundreds of GB RAM for larger trees	High	59
Long-term	Learned value / policy models	Compresses expensive search and improves frontier evaluation.	8–14 weeks	Medium-high if done well	Research-scale; likely multi-GPU or substantial distributed CPU modernization	High	60

A feasible code-level integration map

The good news is that your current repo structure already exposes the right seams.

Improvement	Current files to modify	New files or modules	API and data-structure changes
Combo-level range engine	`poker/analysis/ranges.py`, `poker/analysis/models.py`, `poker/analysis/compose.py`	`poker/analysis/combos.py` or `poker/analysis/range_codec.py` **new, unspecified**	Replace shorthand-only `Range = Dict[str, float]` with a dual representation: shorthand parser plus dense 1326-combo vector. Add helpers for merge, normalize, block board cards, and bucket by position/stack.
Opponent model	`poker/analysis/stats.py`, `poker/engine.py`, `poker/analysis/models.py`	`poker/analysis/opponent_model.py` **new, unspecified**	Add `OpponentModelState` with per-seat priors, update hooks from `action_history`, and `posterior_range(seat, public_state)` methods. Extend `DecisionContext.villain_ranges` from placeholder to required input for strategic bots.

Improvement	Current files to modify	New files or modules	API and data-structure changes
Rollout evaluator	`poker/bots.py`, `poker/engine.py`	`poker/search/rollout.py` **new, unspecified**	Add `ActionEval` structure with `ev_mean`, `ev_stderr`, `win_prob`, `equity`, `latency_ms`. Interface: `evaluate_actions(context, legal_actions, hero_range, villain_ranges, budget)` returning ranked actions.
Solver cache	`poker/analysis/models.py`, `poker/bots.py`	`poker/solver_cache.py` **new, unspecified**	Add `SolverKey` with street, board, pot, effective-stack bucket, position bucket, action-history bucket, size-set ID, and hashes of hero/villain ranges. Value stores action frequencies, EVs, and metadata about solve depth / convergence.
Toy-tree search	`poker/bots.py`, `poker/engine.py`, maybe `poker/analysis/compose.py`	`poker/search/tree.py`, `poker/search/search_bot.py` **new, unspecified**	Use a `SearchPolicy.choose(context, legal_actions)` interface. Leaves call the rollout evaluator or external solver adapter. Respect hard latency budget and fallback to the current `EquityBot` on timeout.
External solver adapter	likely `poker/bots.py` and service layer around engine	`poker/solvers/adapter.py`, `poker/solvers/texassolver.py`, `poker/solvers/pio_upi.py` **new, unspecified**	Use a process-boundary adapter: `solve_subgame(subgame_spec) -> policy`. Serialize board, pot, stacks, range weights, and allowed size set. Avoid in-process AGPL contamination if legal review requires separation.
Benchmark harness	extend `tools/run_llm_simulation.py`	`tools/run_benchmarks.py`, `tools/plot_benchmarks.py` **new, unspecified**	Standardize output schema: matchup name, seed, hand index, street reached, action time, bankroll delta, EV delta, confidence, and bot metadata. Save CSV or Parquet.
Coach explanation alignment	`poker/llm_bot.py`, `poker/ai_coach.py`, `poker/analysis/models.py`	maybe `poker/explanations/renderer.py` **new, unspecified**	Keep LLM as explanation layer. Pass it ranked actions, key EV factors, and concise range summaries; do not let it own primary action logic.

The most important data structures to add

`DecisionContext` is the natural hub. Right now it already carries pot math, stack depth, hand info, players count, human stats, cards, and an optional `villain_ranges`. The strongest low-friction change is to make it truly solver-ready by adding:

- `public_state_key`
- `hero_range`
- `villain_ranges_by_seat`
- `action_history_compact`
- `size_set_id`
- `board_class`
- `range_hashes`
- `solver_cache_hit`
- `latency_budget_ms`

That allows every stronger policy layer — rollouts, search, external solvers, and explanations — to consume the same context object. [61]

A good range representation is just as important. I would keep your existing shorthand parser for developer ergonomics, but internally move to a dense combo-weight vector. Strong open-source solvers and solver docs expose explicit `Range` structures and exploitability utilities because the quality of every downstream EV calculation depends on them. [62]

The rollout API should be separate from search. That keeps things testable. The minimal interface is:

```
evaluate_actions(
    context,
    legal_actions,
    hero_range,
    villain_ranges,
    rollout_budget,
    seed
) -> list[ActionEval]
```

and `ActionEval` should include both mean EV and uncertainty, because many poker matchups are noisy. That one design choice will make your benchmark harness much more honest. This is an architectural recommendation, but it is strongly motivated by the repo's own plan to add MC rollout EV and by how solver libraries expose explicit solve/evaluate hooks. [63]

A realistic hybrid decision flow

The strongest next version of this repository should not be "LLM chooses an action." It should be "solver/search chooses a ranked action set, then the LLM explains it."

```
flowchart TD
    A[Build DecisionContext] --> B[Load combo-level hero and villain ranges]
    B --> C[Apply preflop chart prior or opponent-model posterior]
    C --> D{Solver cache hit?}
```

```
D -- Yes --> E[Return cached policy and EVs]
D -- No --> F{Street / budget / spot}
F -- Preflop simple --> G[Use chart policy plus exploit adjustments]
F -- Postflop small tree --> H[Depth-limited search]
F -- River or solver-backed spot --> I[External subgame solver adapter]
H --> J[Leaf evaluation via rollout EV or value model]
I --> K[Policy frequencies and EVs]
J --> L[Rank actions]
K --> L
G --> L
E --> L
L --> M[Choose policy action]
L --> N[Generate concise explanation payload]
N --> O[Optional LLM explanation renderer]
M --> P[Apply action]
```

This model fits your current file structure, respects latency, preserves deterministic fallbacks, and mirrors the architecture of strong public poker systems much more closely than "prompt the LLM and hope." [64]

Benchmarking, Tooling, Risks, and Timeline

What to measure

You already have one useful metric in the repo: `tools/run_llm_simulation.py` computes total net chips, hands played, and **BB/100**, and can dump per-hand CSV. Keep that, formalize it, and add three more layers: exploitability on toy/subgames, head-to-head matrices, and latency distributions. BB/100 remains the right primary cash-game outcome metric; PokerStars' own learning material defines it as big blinds won per 100 hands and emphasizes that it is meaningful mostly over large samples. [65]

For solver-backed components, add an **exploitability proxy**. Full exploitability in six-player no-limit is not realistic for your whole live game, but it is realistic in toy trees and many postflop or river subgames. OpenSpiel includes exploitability tooling, and `postflop-solver` exposes a `compute_exploitability` function directly. That gives you a rigorous offline quality signal rather than relying only on noisy head-to-head outcomes. [66]

For experiments, I would use four layers.

Experiment	Purpose	Minimum recommendation	Better recommendation
Smoke head-to-head	Catch obvious regressions	5k hands per matchup	10k–20k hands per matchup
Promotion gate	Decide whether a new bot replaces baseline	25k hands across several seeds	50k–100k hands across several seeds
Solver-agreement / exploitability	Validate local search and value approximation	Toy trees or river-only subgames	Flop/turn/river subgame suite by board class

Experiment	Purpose	Minimum recommendation	Better recommendation
Human-facing A/B	Compare coaching or advisor UX	Small internal study	Larger session-based study with action-response logging

Those hand-count recommendations are engineering guidance, not fixed law, but they are consistent with the fact that landmark evaluations were large: DeepStack played tens of thousands of hands against pros, Libratus played on the order of 120,000 hands, Pluribus used 10,000-hand human evaluations, and DecisionHoldem reports roughly 20,000 games against Slumbot. [67]

What the benchmark harness should emit

I would standardize one row per decision and one row per hand.

Per decision:

- policy name
- seed
- hand index
- seat
- street
- board class
- legal action count
- chosen action
- candidate EVs
- latency
- cache hit / miss
- confidence or uncertainty

Per hand:

- final delta in chips
- BB/100 contribution
- all-in EV delta if available
- showdown class
- opponent seat types
- number of runtime fallbacks

That gives you enough to make meaningful visualizations instead of just a final bankroll number.

Visualization suggestions

The best visual layer for this repo is not pretty dashboards first; it is **diagnostic plots**.

I would prioritize:

- **BB/100 confidence intervals by matchup and seed**
- **Latency histograms and P95/P99 per decision policy**
- **Street-by-street action frequency heatmaps**

- **Board-class win-rate summaries** like paired / monotone / coordinated / dry
- **Solver-agreement plots** showing how often the chosen action matches the top solver action
- **Exploitability-vs-iteration curves** for toy-tree solvers
- **Action-selection Sankey or decision-flow graphs** for why a policy reached fold/call/raise

Those are the plots that actually tell you whether a new policy got smarter or just noisier.

Example command lines

The repo already supports one useful simulation command. The README shows loops over `tools/run_llm_simulation.py`, and the script itself outputs aggregate stats and CSVs. A clean baseline command today is: [68]

```
python tools/run_llm_simulation.py \
 --model-alias gpt-5.1-chat-latest \
 --num-hands 1000 \
 --seed 42 \
 --llm-timeout-seconds 5 \
 --csv-output logs
```

I would then add a proposed benchmark harness with command lines like these:

```
python tools/run_benchmarks.py \
 --matchup equity_vs_simple \
 --hands 50000 \
 --seeds 1 2 3 4 5 \
 --csv-output benchmarks/equity_vs_simple.csv
```

```
python tools/run_benchmarks.py \
 --matchup search_vs_equity \
 --hands 20000 \
 --seeds 1 2 3 \
 --latency-budget-ms 150 \
 --rollouts 2048 \
 --csv-output benchmarks/search_vs_equity.csv
```

```
python tools/run_benchmarks.py \
 --round-robin configs/bots.yaml \
 --hands 10000 \
 --seeds 1 2 3 4 \
 --csv-output benchmarks/round_robin.csv
```

and for plotting:

```
python tools/plot_benchmarks.py \
 --input benchmarks/round_robin.csv \
 --out-dir benchmarks/plots
```

Those scripts do not exist yet; I am proposing them because your current repo already has the right `tools/` orientation, logging pattern, and BB/100 output shape. [54]

Libraries and tools to evaluate in priority order

For this repo specifically, the most useful stack to test is:

Priority	Tool	Why
Highest	PokerKit	Already in your repo; keep it as the game engine unless there is a compelling reason not to. It is open-source, pure Python, and built for simulation, hand evaluation, and statistical analysis. [69]
Highest	OpenSpiel	Best path for CFR prototypes, exploitability, universal-poker experiments, and evaluation discipline. [70]
High	`noambrown/poker_solver`	Modern, MIT-licensed, modular river solver with Python/C++ split and JSON subgame format. [46]
High	TexasSolver	Fast postflop engine with cross-language calls and JSON strategy export; very attractive if AGPL is acceptable. [71]
High	`postflop-solver`	Cleaner library surface than a GUI app; exposes exploitability and range/game structures directly. [62]
Medium	PioSOLVER via UPI	Commercial, but the fastest route to strong postflop advice if you are open to non-open-source dependencies; UPI is specifically meant for external-tool integration. [72]
Medium	RLCard	Good for RL baselines and offline experiments, less ideal as the live engine because of action abstraction mismatch. [73]
Lower	PokerRL / Deep-CFR	Worth it only if you commit to a research-heavy learned-policy track; otherwise too much infra drag. [74]
Lower	PyPokerEngine	Useful educationally, but likely redundant because PokerKit is already in place. [40]

Risks, ethics, and legal constraints

The biggest engineering-legal risk is **licensing**, not poker theory. OpenSpiel, RLCard, PyPokerEngine, PokerRL, Deep-CFR, Slumbot2019, Noam Brown's river solver, and your repo itself are under Apache or MIT-style licenses, which are integration-friendly. TexasSolver, `postflop-solver`, and DecisionHoldem are AGPL-licensed, which makes them much riskier to embed directly in a networked application without counsel and a deployment strategy. [75]

The biggest product-policy risk is **real-money or third-party platform misuse**. Major poker operators explicitly prohibit bots and real-time assistance. PokerStars' terms prohibit AI, bots, and tools that execute or assist actions for unfair advantage, and GGPoker's Security & Ecology Policy says a human

must decide the action and exact bet/raise size and that tools reading the game state and providing decision aid are prohibited. If this repo is ever used outside a private sandbox or internal research environment, platform terms must be reviewed case by case. [76]

There is also a compute-risk staircase. Short-term upgrades such as combo ranges, opponent models, and benchmark harnesses are cheap. Rollouts and toy-tree search are moderately expensive but manageable on a normal multi-core workstation. Blueprint-style offline solving is where costs spike hard: DecisionHoldem's own documentation says its blueprint was trained with linear CFR on a 48-core workstation for 3–4 days, and its development environment note cites 512 GB of RAM. That is exactly the kind of resource wall you should postpone until earlier, cheaper improvements are already paying off. [47]

The final practical risk is tech debt from research code. PokerRL and Deep-CFR are useful and respected, but they are anchored to older infrastructure choices: Docker, PyCrayon, Ray-based distributed runs, and old PyTorch versions. They are excellent reference material, but lifting them into a modern product repo can cost more than building a much smaller modernized value-network stack yourself. [77]

A realistic roadmap

```
gantt
    title Suggested 6–12 month roadmap for upgrading poker-coach-alpha
    dateFormat  YYYY-MM-DD
    axisFormat  %b %Y

    section Foundation
    Benchmark harness and telemetry           :a1, 2026-05-01, 21d
    Combo-level ranges and chart priors       :a2, 2026-05-15, 28d
    Opponent model and range updates          :a3, 2026-06-01, 28d

    section Stronger local policy
    Rollout EV evaluator                      :b1, 2026-06-15, 35d
    Search bot with latency budget            :b2, 2026-07-15, 42d
    Solver cache and public-state keys        :b3, 2026-08-01, 28d

    section External solving
    River adapter integration                 :c1, 2026-08-15, 28d
    Postflop solver adapter evaluation        :c2, 2026-09-01, 42d

    section Learning path
    Offline blueprint experiments             :d1, 2026-10-01, 56d
    Value / policy model prototype            :d2, 2026-11-01, 70d
    Six-max blueprint feasibility review      :d3, 2026-12-15, 28d
```

If you follow that path, the project evolves in the right order: **first honest measurement, then better ranges, then stronger local search, then solver integration, then learned compression**. That sequence matches both the repo's own stated roadmap and the logic of mainstream poker AI development. [78]

The most important strategic judgment in this whole report is probably the simplest one: **treat the current repo as a product shell waiting for a stronger decision engine, not as a strategy engine that**

merely needs tuning. Once you make that mental shift, the right next steps become much easier to prioritize.

---

1 6 11 14 23 34 36 68 https://github.com/Wangmengguo/poker-coach-alpha
https://github.com/Wangmengguo/poker-coach-alpha

2 16 18 20 22 25 27 29 35 56 60 67 https://www.science.org/doi/epdf/10.1126/science.aam6960
https://www.science.org/doi/epdf/10.1126/science.aam6960

3 19 53 https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/poker/analysis/stats.py
https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/poker/analysis/stats.py

4 21 55 63 78 https://github.com/Wangmengguo/poker-coach-alpha/blob/main/PLAN.md?plain=1
https://github.com/Wangmengguo/poker-coach-alpha/blob/main/PLAN.md?plain=1

5 51 https://www.science.org/doi/pdf/10.1126/science.aay2400?download=true
https://www.science.org/doi/pdf/10.1126/science.aay2400?download=true

7 8 15 33 https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/bots.py?plain=1
https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/bots.py?plain=1

9 17 https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/engine.py?plain=1
https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/engine.py?plain=1

10 13 52 https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/poker/analysis/ranges.py
https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/poker/analysis/ranges.py

12 69 https://pokerkit.readthedocs.io/en/
https://pokerkit.readthedocs.io/en/

24 44 49 58 71 https://github.com/bupticybee/TexasSolver
https://github.com/bupticybee/TexasSolver

26 https://poker.cs.ualberta.ca/publications/NIPS07-cfr.pdf
https://poker.cs.ualberta.ca/publications/NIPS07-cfr.pdf

28 https://proceedings.neurips.cc/paper_files/paper/2009/file/00411460f7c92d2124a67ea0f4cb5f85-Paper.pdf
https://proceedings.neurips.cc/paper_files/paper/2009/file/00411460f7c92d2124a67ea0f4cb5f85-Paper.pdf

30 https://eprints.whiterose.ac.uk/id/eprint/75048/1/CowlingPowleyWhitehouse2012.pdf
https://eprints.whiterose.ac.uk/id/eprint/75048/1/CowlingPowleyWhitehouse2012.pdf

31 https://arxiv.org/abs/2007.13544
https://arxiv.org/abs/2007.13544

32 https://arxiv.org/abs/1811.00164
https://arxiv.org/abs/1811.00164

37 38 75 https://github.com/google-deepmind/open_spiel
https://github.com/google-deepmind/open_spiel

39 73 https://github.com/datamllab/rlcard
https://github.com/datamllab/rlcard

40 https://github.com/ishikota/PyPokerEngine
https://github.com/ishikota/PyPokerEngine

41 50 74 77 https://github.com/EricSteinberger/PokerRL
https://github.com/EricSteinberger/PokerRL

42 https://github.com/EricSteinberger/Deep-CFR
https://github.com/EricSteinberger/Deep-CFR

43 https://github.com/ericgjackson/slumbot2019
https://github.com/ericgjackson/slumbot2019

45 https://github.com/b-inary/postflop-solver
https://github.com/b-inary/postflop-solver

46 https://github.com/noambrown/poker_solver
https://github.com/noambrown/poker_solver

47 59 https://github.com/ai-decision/decisionholdem
https://github.com/ai-decision/decisionholdem

48 66 70 https://openspiel.readthedocs.io/en/stable/_sources/algorithms.md.txt
https://openspiel.readthedocs.io/en/stable/_sources/algorithms.md.txt

54 65 https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/tools/run_llm_simulation.py
https://raw.githubusercontent.com/Wangmengguo/poker-coach-alpha/main/tools/run_llm_simulation.py

57 https://www.science.org/doi/epdf/10.1126/science.aao1733
https://www.science.org/doi/epdf/10.1126/science.aao1733

61 https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/analysis/models.py?plain=1
https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/analysis/models.py?plain=1

62 https://b-inary.github.io/postflop_solver/postflop_solver/
https://b-inary.github.io/postflop_solver/postflop_solver/

64 https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/llm_bot.py?plain=1
https://github.com/Wangmengguo/poker-coach-alpha/blob/main/poker/llm_bot.py?plain=1

72 https://piosolver.com/docs/upi/
https://piosolver.com/docs/upi/

76 https://www.pokerstars.com/tos/
https://www.pokerstars.com/tos/